Classic + Simple

Code guide for `TeleCon_Classic_Simple`

ESP32 DevKit · Bluetooth Classic · text lines

Version 1.0 2026

Abstract

This note is the Arduino sketch in the folder `Classic_Simple/TeleCon_Classic_Simple/`. The phone setting is Classic + Simple. The board name is `ESP32-TC-RC-BT-Simple`. The handshake is `RC:CONNECT,proto,simple`. The declarations of each file come first: includes, macros, variables, and types. Then each function, with its code in the IDE colors. The two complete files are the annexes.

IDE color: `void`, `if`, `#define`, `Serial.print`, `"text"`, `BT_NAME`, `// comment`.

Contents

1 File declarations **2**

1.1 In `TelemetryChannels.h` . . . 2

1.2 In `TeleCon_Classic_Simple.ino` . . . 3

2 What each function does **6**

2.1 In `TeleCon_Classic_Simple.ino` . . . 6

2.2 In `TelemetryChannels.h` . . . 18

Annex A `TeleCon_Classic_Simple.ino` **20**

Annex B `TelemetryChannels.h` **28**

1 File declarations

This is the head of each file: what the compiler sees before the first function body. The code uses the IDE colors. The line numbers are the line numbers of the file.

1.1 In `TelemetryChannels.h`

The `.ino` includes this header, so these names exist before `kTelemetryChannels[]`.

```
1  #pragma once
2
3  #include <Arduino.h>
4
5  /*
6   * Phone CH1...CH8 are RC:PLOT v0...v7.
7   * Edit kTelemetryChannels[] in the sketch to choose what each channel
   reports (0-255).
8   *
9   * TEL_VEHICLE  built-in signal (temp, throttle, battery, steer, lights,
   knobs)
10  * TEL_ADC      analogRead(pin), 0-4095 scaled to 0-255
11  * TEL_GPIO     digitalRead(pin) → 0 or 255 (an output pin reports what was
   written)
12  * TEL_PWM      LEDC duty on pin. dutyMin maps to 0, dutyMax maps to 255.
13  *              8-bit motor PWM uses 0...255. 12-bit servo travel 1000-2000 µ
   s uses 204...409.
14  *              Attach that pin with ledcAttach / ledcSetup before it is read
   .
15  */
```

`#pragma once` brings the file into the build once, even if something includes it twice. `Arduino.h` brings `uint8_t`, `analogRead`, `digitalRead`, and `ledcRead`. The comment in the middle generates no code: CH1 is `v0`, CH8 is `v7`, and each sample is a byte 0–255.

```
17 enum TelKind : uint8_t {
18   TEL_VEHICLE = 0,
19   TEL_ADC,
20   TEL_GPIO,
21   TEL_PWM,
22 };
23
24 enum TelVehicle : uint8_t {
25   TEL_TEMP = 0,
26   TEL_THROTTLE,
27   TEL_BATT,
28   TEL_STEER,
29   TEL_LIGHTS,
30   TEL_KNOB_L,
31   TEL_KNOB_R,
32   TEL_RY,
33 };
```

`TelKind` says where a row's byte comes from. `TEL_VEHICLE` calls `telemetryVehicle()`. `TEL_ADC`

reads an analog pin. TEL_GPIO reads a digital pin. TEL_PWM reads an LEDC duty. The numeric value starts at 0 and continues in order.

TelVehicle names the built-in signal, only when the row is TEL_VEHICLE. TEL_TEMP is temperature, TEL_THROTTLE is the throttle, TEL_BATT is the battery, TEL_STEER is steering, TEL_LIGHTS is the lights, TEL_KNOB_L and TEL_KNOB_R are the knobs, and TEL_RY is the right stick's Y axis. The factory table does not use TEL_RY.

```
35 struct TelChannel {
36   TelKind kind;
37   TelVehicle vehicle;
38   int pin;
39   int ledcChannel;
40   int dutyMin;
41   int dutyMax;
42 };
43
44 using TelVehicleFn = uint8_t (*)(TelVehicle id);
```

TelChannel is one row of the table. kind is the TelKind. vehicle is the TelVehicle; an ADC, GPIO, or PWM row does not read that field. pin is the GPIO, or −1 when the row has no pin. ledcChannel is the LEDC channel on core 2.x, or −1 when there is no PWM. dutyMin and dutyMax are the ends of the duty range: the minimum is sent as 0 and the maximum as 255.

TelVehicleFn is the type of the pointer to telemetryVehicle(): it takes a TelVehicle and returns a byte.

1.2 In TeleCon_Classic_Simple.ino

```
1  /*
2   * TeleCon RC Vehicle Pro - Classic Simple (ESP32 Dev Module, no camera)
3   *
4   * Android (TeleCon4ESP32):
5   *   1. Pair Bluetooth "ESP32-TC-RC-BT-Simple" (PIN 1234 if asked)
6   *   2. Control Panel or RC Vehicle Pro → Board DevKit → Classic + Simple
7   *   3. Connect - phone sends RC:CONNECT,proto,simple
8   *
9   * Single-file starter for Normal users. Edit onControl() / onButton().
10  * Advanced Classic Binary: ../../Classic_Binary/
11  */
12
13 #include "BluetoothSerial.h"
14 #include "TelemetryChannels.h"
```

The comment block does not enter the binary. It states the Bluetooth name, the PIN, and the handshake line. BluetoothSerial.h is the Classic port. TelemetryChannels.h is the header above.

```
16 // === CONFIG ===
17 #define BT_NAME             "ESP32-TC-RC-BT-Simple"
18 #define BT_PIN              "1234"
19 #define APP_PREFIX          "RC"
20 #define PROTO_WIRE          "simple"
21 #define SERIAL_BAUD         115200
```

```
22
23 // 1 = Serial banners + stick/HS/IO logs. Set 0 when Connect works.
24 #define ENABLE_DEBUG        1
25
26 #if ENABLE_DEBUG
27   #define DBG_PRINT(x)      Serial.print(x)
28   #define DBG_PRINTLN(x)    Serial.println(x)
29   #define DBG_PRINTF(...)   Serial.printf(__VA_ARGS__)
30 #else
31   #define DBG_PRINT(x)      ((void)0)
32   #define DBG_PRINTLN(x)    ((void)0)
33   #define DBG_PRINTF(...)   ((void)0)
34 #endif
35
36 #define TELEMETRY_DATA_MS   500
37 #define ENABLE_PLOT         1
38 #define PLOT_MS             50
39
40 #define ENABLE_SERIAL_INJECT 1
41 #define SERIAL_MAX_LINE     192
42
43 // Demo HUD values (replace with ADC reads when you wire sensors)
44 #define DEMO_MOTOR_TEMP     90
45 #define DEMO_BATT_PCT       76
```

BT_NAME is the name Android sees when pairing. BT_PIN is 1234. APP_PREFIX has to be RC and PROTO_WIRE has to be simple: those are the two strings handleConnect() accepts. SERIAL_BAUD is 115200 for the monitor.

ENABLE_DEBUG at 1 writes to the monitor. DBG_PRINT becomes Serial.print. DBG_PRINTLN becomes Serial.println. DBG_PRINTF becomes Serial.printf. At 0 those three macros become empty and the link stays the same.

TELEMETRY_DATA_MS is the dashboard period, 500 ms. ENABLE_PLOT at 1 sends RC:PLOT. PLOT_MS is 50 ms. ENABLE_SERIAL_INJECT at 1 lets you paste a line into the monitor after the handshake. SERIAL_MAX_LINE cuts that line at 192 characters.

DEMO_MOTOR_TEMP is 90 and DEMO_BATT_PCT is 76. They are fixed dashboard numbers until a sensor is fitted. They do not come from a pin.

```
47 // === BLUETOOTH ===
48 BluetoothSerial SerialBT;
49
50 bool handshakeOk = false;
51 bool btWasConnected = false;
52
53 int g_lx = 0, g_ly = 0, g_rx = 0, g_ry = 0;
54 int g_lk = 512, g_rk = 512;
55 uint8_t g_sw = 0;
56
57 unsigned long lastDataMs = 0;
58 unsigned long lastPlotMs = 0;
59
60 #if ENABLE_SERIAL_INJECT
61 static String serialLineBuf;
62 #endif
```

```
63
64 static uint16_t lastPrintLX = 0xFFFF;
65 static uint16_t lastPrintLY = 0xFFFF;
66 static uint16_t lastPrintRX = 0xFFFF;
67 static uint16_t lastPrintRY = 0xFFFF;
68 static uint16_t lastPrintKnobL = 0xFFFF;
69 static uint16_t lastPrintKnobR = 0xFFFF;
70 static uint8_t lastPrintSw = 0xFF;
```

SerialBT is the Classic port. handshakeOk is the gate: while it is false there is no onControl(), no RC:DATA, and no RC:PLOT. btWasConnected stores the client from the previous pass, so a connect or a drop is noticed once.

g_lx, g_ly, g_rx, and g_ry are the sticks, from −100 to 100. They start at 0. g_ly is the throttle and g_rx is steering. g_lk and g_rk are the knobs, from 0 to 1023, and they start at 512. g_sw is the switch byte. Bit 0 is the lights.

lastDataMs marks the dashboard. lastPlotMs marks the plot. Both store millis(). serialLineBuf gathers the line pasted into the monitor. It exists only when injection is compiled in. The lastPrint* values store the last printed image of each axis, knob, and switch. They start at 0xFFFF or 0xFF so the first sample is always printed. They are not sent to the phone.

```
72 String getValue(const String& line, const char* key);
73 void handleLine(const String& line);
74 void handleConnect(const String& app, const String& line);
75 void sendLine(const String& line);
76 void resetHandshake();
77 void applyFailSafe();
78 void debugPrintControl(int lx, int ly, int rx, int ry, int lk, int rk,
   uint8_t sw);
79 void debugPrintButton(int id);
80 uint16_t stickTo12(int s);
81 #if ENABLE_SERIAL_INJECT
82 void serialInjectPoll();
83 void handleSerialInjectLine(const String& line);
84 #endif
```

These lines are the compiler's notice that each body comes further down, so setup() and loop() can call them. serialInjectPoll and handleSerialInjectLine are declared only when ENABLE_SERIAL_INJECT is 1. Each body is in the next section.

```
87 // === STEERING SERVO ===
88 // DevKit: set ENABLE_STEER_SERVO 1 and wire a servo on PIN_STEER_SERVO.
89 // rx = 0 is always 1500 µs. Phone trim already offsets rx before it is sent.
90 #ifndef ENABLE_STEER_SERVO
91 #define ENABLE_STEER_SERVO 1
92 #endif
93 #ifndef PIN_STEER_SERVO
94 #define PIN_STEER_SERVO 13
95 #endif
96 #ifndef CH_STEER_SERVO
97 #define CH_STEER_SERVO 2
98 #endif
```

ENABLE_STEER_SERVO at 1 arms the servo. At 0 the two servo functions stay empty.

PIN_STEER_SERVO is GPIO 13. CH_STEER_SERVO is LEDC channel 2, the one core 2.x uses. The #ifndef lets the IDE change the value before this file, and uses the factory value when nobody changed it.

```
100 // Phone CH1...CH8. Change a row to TEL_ADC, TEL_GPIO, or TEL_PWM and set the
     pin.
101 // Examples:
102 //   { TEL_ADC,  TEL_TEMP,  34, -1, 0, 1 },            // analog pin 34
103 //   { TEL_GPIO, TEL_LIGHTS, 15, -1, 0, 1 },           // digital pin 15
104 //   { TEL_PWM,  TEL_THROTTLE, 25, 0, 0, 255 },        // 8-bit motor PWM on
     GPIO 25
105 static const TelChannel kTelemetryChannels[8] = {
106   { TEL_VEHICLE, TEL_TEMP,     -1,              -1,   0,   1 },
107   { TEL_VEHICLE, TEL_THROTTLE, -1,              -1,   0,   1 },
108   { TEL_VEHICLE, TEL_BATT,     -1,              -1,   0,   1 },
109   { TEL_VEHICLE, TEL_STEER,    -1,              -1,   0,   1 },
110   { TEL_VEHICLE, TEL_LIGHTS,   -1,              -1,   0,   1 },
111   { TEL_VEHICLE, TEL_KNOB_L,   -1,              -1,   0,   1 },
112   { TEL_VEHICLE, TEL_KNOB_R,   -1,              -1,   0,   1 },
113   { TEL_PWM,     TEL_STEER,    PIN_STEER_SERVO, CH_STEER_SERVO, 204, 409 },
114 };
```

kTelemetryChannels has eight rows and does not accept a ninth. Each brace is a TelChannel: kind, signal, pin, LEDC channel, dutyMin, and dutyMax. Row 0 is CH1 (v0) and row 7 is CH8 (v7).

From the factory, rows 0 to 6 are TEL_VEHICLE: temperature, throttle, battery, steering, lights, left knob, and right knob. The pin and the channel are −1 because the signal does not come from a GPIO. Row 7 is the servo's TEL_PWM: pin PIN_STEER_SERVO, channel CH_STEER_SERVO, duty from 204 to 409 (the 1000–2000 µs travel).

```
139 constexpr int kSteerCenterUs = 1500;
140 constexpr int kSteerTravelUs = 500;
```

These two constants sit just before steerDutyFromUs(). They are outside the header. kSteerCenterUs is 1500 µs, the center. kSteerTravelUs is 500 µs to each side of the stick. rx = 0 stays at 1500 µs.

2 What each function does

The order follows each file, after the declarations. Under each signature is the whole function, in the IDE colors. The text uses the same coloring: orange is a function, from the sketch or from a library (onControl(), print, ledcWrite), and purple is a name from this project (BT_NAME, g_lx, TEL_TEMP).

DBG_PRINT, DBG_PRINTLN, and DBG_PRINTF are macros. With ENABLE_DEBUG at 1 they write to the serial monitor. At 0 the compiler removes them.

2.1 In TeleCon_Classic_Simple.ino

`uint8_t telemetryVehicle(TelVehicle id)`

```
116 uint8_t telemetryVehicle(TelVehicle id) {
117   switch (id) {
118     case TEL_TEMP:
```

```
119       return (uint8_t)DEMO_MOTOR_TEMP;
120     case TEL_THROTTLE:
121       return (uint8_t)constrain(map(abs(g_ly), 0, 100, 0, 255), 0, 255)
;
122     case TEL_BATT:
123       return (uint8_t)constrain(map(DEMO_BATT_PCT, 0, 100, 0, 255), 0,
255);
124     case TEL_STEER:
125       return (uint8_t)constrain(map(abs(g_rx), 0, 100, 0, 255), 0, 255)
;
126     case TEL_LIGHTS:
127       return (g_sw & 0x01) ? 255 : 0;
128     case TEL_KNOB_L:
129       return (uint8_t)map(constrain(g_lk, 0, 1023), 0, 1023, 0, 255);
130     case TEL_KNOB_R:
131       return (uint8_t)map(constrain(g_rk, 0, 1023), 0, 1023, 0, 255);
132     case TEL_RY:
133       return (uint8_t)constrain(map(abs(g_ry), 0, 100, 0, 255), 0, 255)
;
134     default:
135       return 0;
136   }
137 }
```

Line 116. Returns the byte 0–255 of a built-in signal, the one a `TEL_VEHICLE` row asks for. `TEL_TEMP` is the constant `DEMO_MOTOR_TEMP`. `TEL_THROTTLE` is the magnitude of `g_ly`. `TEL_BATT` scales `DEMO_BATT_PCT` from 0–100 to 0–255. `TEL_STEER` is the magnitude of `g_rx`. `TEL_LIGHTS` is 255 when bit 0 of `g_sw` is set, and 0 when it is clear. `TEL_KNOB_L` and `TEL_KNOB_R` map the knob from 0–1023 to 0–255. `TEL_RY` is the magnitude of `g_ry`. Any other `id` returns 0. Throttle, steering, and `TEL_RY` drop the sign: both ends of the stick produce the same byte.

```
142 uint32_t steerDutyFromUs(int us) {
143   int pulse = constrain(us, 500, 2500);
144   return (uint32_t)((long)pulse * 4095L /
 20000L);
145 }
```

uint32_t steerDutyFromUs(int us)

Line 142. Turns a pulse width, in microseconds, into a 12-bit LEDC duty: pulse $\times 4095/20000$. It first limits the pulse to 500–2500 µs. 1500 µs lands at the center of a 20 ms frame.

```
147 uint32_t steerDutyFromRx(int rx) {
148   int s = constrain(rx, -100, 100);
149   int us = (int)map(s, -100, 100,
kSteerCenterUs - kSteerTravelUs,
kSteerCenterUs + kSteerTravelUs);
150   return steerDutyFromUs(us);
151 }
```

uint32_t steerDutyFromRx(int rx)

Line 147. Limits `rx` to $-100 \ldots 100$, turns it into microseconds (0 is 1500 µs, full travel is ±500 µs), and calls `steerDutyFromUs`. The phone trim is already inside `rx`.

void steerServoBegin()

```
153 #if ENABLE_STEER_SERVO
154 void steerServoBegin() {
155 #if ESP_ARDUINO_VERSION_MAJOR >= 3
156   ledcAttach(PIN_STEER_SERVO, 50, 12);
157   ledcWrite(PIN_STEER_SERVO, steerDutyFromUs(kSteerCenterUs));
158 #else
159   ledcSetup(CH_STEER_SERVO, 50, 12);
160   ledcAttachPin(PIN_STEER_SERVO, CH_STEER_SERVO);
161   ledcWrite(CH_STEER_SERVO, steerDutyFromUs(kSteerCenterUs));
162 #endif
163 }
```

```
174 void steerServoBegin() {}
```

Line 154, or line 174 when `ENABLE_STEER_SERVO` is 0. With the servo on, it sets a 50 Hz, 12-bit PWM on `PIN_STEER_SERVO` and writes the center. On core 3.x the channel is attached by pin number (`ledcAttach`, `ledcWrite`). On 2.x it uses `CH_STEER_SERVO` (`ledcSetup`, `ledcAttachPin`, `ledcWrite`). With the switch at 0 the function does nothing.

void steerServoWriteRx(int rx)

```
165 void steerServoWriteRx(int rx) {
166   uint32_t duty = steerDutyFromRx(rx);
167 #if ESP_ARDUINO_VERSION_MAJOR >= 3
168   ledcWrite(PIN_STEER_SERVO, duty);
169 #else
170   ledcWrite(CH_STEER_SERVO, duty);
171 #endif
172 }
```

```
175 void steerServoWriteRx(int rx) { (void)rx; }
```

Line 165, or line 175 when the servo is off. It computes the duty for `rx` and writes it. The empty version only discards `rx`, so the compiler stays quiet about an unused parameter. `applyFailSafe()` calls it with 0, which is the center.

void onControl(int lx, int ly, int rx, int ry, int lk, int rk, uint8_t sw)

```
178 void onControl(int lx, int ly, int rx, int ry, int lk, int rk, uint8_t
sw) {
179   debugPrintControl(lx, ly, rx, ry, lk, rk, sw);
180
181   // rx = 0 → 1500 µs. Trim ticks on the phone already shift this
value.
182   steerServoWriteRx(rx);
183
184   // YOUR CODE HERE - drive motors / other actuators from sticks &
knobs
185   //   ly = throttle (-100..100), rx = steering, rk = pan, sw bit0 =
lights
```

```
  (void)lx; (void)ly; (void)ry; (void)lk; (void)rk; (void)sw;
}
```

Line 178. Runs once for each `RC:CTRL` line. It prints the change on the monitor, moves the servo with `rx`, and leaves the `YOUR CODE HERE` block for whatever should follow the stick: motor PWM, a pan servo from `rk`, a lights pin from `sw` & 0x01. The `(void)` casts silence the warning for values the starter sketch does not use yet. Remove the cast of a value when you start using it.

void onButton(int id)

```
void onButton(int id) {
  debugPrintButton(id);

  // YOUR CODE HERE - button actions
  if (id == 2) {
    // Horn note - pulse buzzer briefly
  } else if (id == 4) {
    // Center pan note - move camera pan servo to center
  }
}
```

Line 189. Runs once for each `RC:BTN`. It prints the id. The factory comment leaves id 2 for a short horn pulse and id 4 for centering a pan servo. Those two bodies are empty. A button that must stay down belongs as a bit of `sw` inside `onControl()`, because `RC:BTN` is the press, and the held level travels in `RC:CTRL`.

void setup()

```
void setup() {
  Serial.begin(SERIAL_BAUD);
  delay(200);

  steerServoBegin();

  // Legacy PIN pairing only (Android often fails if SSP + setPin are
mixed).
  SerialBT.disableSSP();
  SerialBT.setPin(BT_PIN, 4);

  // isMaster=false, disableBLE=true → Classic SPP only
  if (!SerialBT.begin(BT_NAME, false, true)) {
    Serial.println("Bluetooth init failed!");
    while (true) { delay(1000); }
  }

  DBG_PRINTLN();
  DBG_PRINTLN("=== TeleCon RC Vehicle Pro - Classic Simple ===");
  DBG_PRINT("BT name: ");
  DBG_PRINTLN(BT_NAME);
  DBG_PRINTLN("Pair with PIN 1234 if the phone asks.");
  DBG_PRINTLN("Android: RC Vehicle Pro → Board DevKit → Normal →
```

```
Classic + Simple");
222   DBG_PRINTLN("Handshake: RC:CONNECT,proto,simple  →  RC:ACK,app,RC");
223   DBG_PRINTLN("Examples:");
224   DBG_PRINTLN("  RC:CONNECT,proto,simple");
225   DBG_PRINTLN("  RC:CTRL,lx,0,ly,0,rx,0,ry,0,lk,512,rk,512,sw,00");
226   DBG_PRINTLN("  RC:BTN,id,1");
227   DBG_PRINTLN("Steer: rx=0 is 1500 µs. Trim stays on the phone.");
228   DBG_PRINTLN("Edit onControl() / onButton() at YOUR CODE HERE.");
229   DBG_PRINTLN("Telemetry: RC:DATA every 500 ms (speed×10 from |ly|,
demo batt/temp).");
230 #if ENABLE_PLOT
231   DBG_PRINTLN("Plot ON - RC:PLOT ~50 ms, CH1...CH8 (edit
kTelemetryChannels[]).");
232 #else
233   DBG_PRINTLN("Plot OFF - set ENABLE_PLOT 1 to send RC:PLOT.");
234 #endif
235 #if ENABLE_SERIAL_INJECT
236   DBG_PRINTLN("Serial inject @ 115200 (after handshake, line ending =
Newline):");
237   DBG_PRINTLN("  RC:DATA,left,186,right,0,lo,1,ro,0,lg,1,rg,0,analog
,90,batt,76,led,01");
238   DBG_PRINTLN("  RC:PLOT,v0,128,v1,200,v2,64,v3,180,v4,0,v5,0,v6,0,v7,0
");
239   DBG_PRINTLN("  Inject is one-shot (pulse); auto telemetry resumes
after.");
240 #endif
241   DBG_PRINTLN();
242 }
```

Line 200. Starts serial at 115200, waits 200 ms, centers the servo, turns Secure Simple Pairing off, sets the PIN `1234`, and opens Bluetooth Classic with `BT_NAME`, as a slave and without BLE. If `SerialBT.begin` fails, it prints the failure and stays in `while (true)`. The banner after that is debug only: the name, the handshake, and sample lines for the monitor.

void loop()

```
244 void loop() {
245   bool connected = SerialBT.hasClient();
246   if (connected != btWasConnected) {
247     btWasConnected = connected;
248     if (connected) {
249       resetHandshake();
250       DBG_PRINTLN("[BT] connected - waiting for RC:CONNECT,proto,simple
");
251     } else {
252       applyFailSafe();
253       resetHandshake();
254       DBG_PRINTLN("[BT] disconnected");
255     }
256   }
257
258   while (SerialBT.available()) {
```

```
    String line = SerialBT.readStringUntil('\n');
    line.trim();
    if (line.length() > 0) {
      handleLine(line);
    }
  }

#if ENABLE_SERIAL_INJECT
  serialInjectPoll();
#endif

  if (!handshakeOk) return;

  unsigned long now = millis();

  if (now - lastDataMs >= TELEMETRY_DATA_MS) {
    lastDataMs = now;

    int speedX10 = abs(g_ly) * 25 / 10;
    int analog = DEMO_MOTOR_TEMP;
    int batt = DEMO_BATT_PCT;

    char data[128];
    snprintf(data, sizeof(data),
      "RC:DATA,left,%d,right,0,lo,1,ro,0,lg,1,rg,0,analog,%d,batt,%d,
led,%02X",
      speedX10, analog, batt, g_sw);
    sendLine(data);
  }

#if ENABLE_PLOT
  if (now - lastPlotMs >= PLOT_MS) {
    lastPlotMs = now;

    uint8_t sample[8];
    telemetryReadAll(sample, kTelemetryChannels, telemetryVehicle);
    char plot[128];
    snprintf(plot, sizeof(plot),
      "RC:PLOT,v0,%u,v1,%u,v2,%u,v3,%u,v4,%u,v5,%u,v6,%u,v7,%u",
      sample[0], sample[1], sample[2], sample[3],
      sample[4], sample[5], sample[6], sample[7]);
    sendLine(plot);
  }
#endif
}
```

Line 244. Arduino calls `loop()` again and again, from the moment `setup()` finishes. Each pass follows these steps, and the next one starts when the previous one has finished.

```
245   bool connected = SerialBT.hasClient();
246   if (connected != btWasConnected) {
247     btWasConnected = connected;
248     if (connected) {
249       resetHandshake();
250       DBG_PRINTLN("[BT] connected -
waiting for RC:CONNECT,proto,simple");
251     } else {
252       applyFailSafe();
253       resetHandshake();
254       DBG_PRINTLN("[BT] disconnected");
255     }
256   }
```

1. Bluetooth client

Reads `SerialBT.hasClient()` and compares it with `btWasConnected`. If it changed, it stores the new state. If the phone just arrived, it calls `resetHandshake()`: `handshakeOk` stays false, and the monitor says it is waiting for `RC:CONNECT,proto,simple`. If the phone just left, it calls `applyFailSafe()` (sticks at 0, knobs at 512, switches at 0, servo at center) and then `resetHandshake()`. If the client is unchanged, this block calls nobody.

```
258   while (SerialBT.available()) {
259     String line = SerialBT.
readStringUntil('\n');
260     line.trim();
261     if (line.length() > 0) {
262       handleLine(line);
263     }
264   }
```

2. Bluetooth lines

While `SerialBT` has bytes, it reads up to `\n`, strips spaces with `trim()`, and, if text remains, calls `handleLine()`. A valid `RC:CONNECT,proto,simple` line in this same pass sets `handshakeOk` true before the timers below.

```
266 #if ENABLE_SERIAL_INJECT
267   serialInjectPoll();
268 #endif
```

3. Serial monitor

With `ENABLE_SERIAL_INJECT` at 1, it calls `serialInjectPoll()`. A monitor line that starts with `RC:DATA` or with `RC:PLOT` goes out to the phone at that moment. With the switch at 0, this block is left out of the binary.

```
270   if (!handshakeOk) return;
```

4. Handshake gate

If `handshakeOk` is false, `loop()` ends at that `return`. The dashboard and the plot wait for the answer from `handleConnect()`: `RC:ACK,app,RC`.

```
272   unsigned long now = millis();
```

5. Clock

Stores `millis()` in `now`. The dashboard and the plot use this same instant, each with its own mark. Meeting one deadline does not restart the other. Both can come due on the same pass: the dashboard first, then the plot.

```
  if (now - lastDataMs >= TELEMETRY_DATA_MS) {
    lastDataMs = now;

    int speedX10 = abs(g_ly) * 25 / 10;
    int analog = DEMO_MOTOR_TEMP;
    int batt = DEMO_BATT_PCT;

    char data[128];
    snprintf(data, sizeof(data),
      "RC:DATA,left,%d,right,0,lo,1,ro,0,lg,1,rg,0,analog,%d,batt,%d,led,%02X",
      speedX10, analog, batt, g_sw);
    sendLine(data);
  }
```

6. Dashboard, every TELEMETRY_DATA_MS (500 ms)

When now − lastDataMs reaches the period, it stores now in lastDataMs and builds an RC:DATA line. left is abs(g_ly) * 25 / 10: throttle 100 goes out as 250, and the phone reads it as speed ×10. abs() sends the magnitude, so full reverse throttle also goes out as 250. right stays fixed at 0. lo,1,ro,0,lg,1,rg,0 are the panel lamps, also fixed. analog copies DEMO_MOTOR_TEMP (90) and batt copies DEMO_BATT_PCT (76). led is g_sw in two hexadecimal digits. sendLine() sends that line.

```
#if ENABLE_PLOT
  if (now - lastPlotMs >= PLOT_MS) {
    lastPlotMs = now;

    uint8_t sample[8];
    telemetryReadAll(sample, kTelemetryChannels, telemetryVehicle);
    char plot[128];
    snprintf(plot, sizeof(plot),
      "RC:PLOT,v0,%u,v1,%u,v2,%u,v3,%u,v4,%u,v5,%u,v6,%u,v7,%u",
      sample[0], sample[1], sample[2], sample[3],
      sample[4], sample[5], sample[6], sample[7]);
    sendLine(plot);
  }
#endif
```

7. Plot, every PLOT_MS (50 ms)

This block enters the binary when ENABLE_PLOT is 1. When now − lastPlotMs reaches the period, it stores now in lastPlotMs, calls telemetryReadAll() with kTelemetryChannels and telemetryVehicle, and sends RC:PLOT from v0 to v7, in the order of the eight rows. With ENABLE_PLOT at 0, this block is left out of the binary.

void handleSerialInjectLine(const String& line)

```
#if ENABLE_SERIAL_INJECT
void handleSerialInjectLine(const String& line) {
  if (!SerialBT.hasClient()) {
    DBG_PRINTLN("[SERIAL] no BT client - line not sent");
    return;
  }
  if (!handshakeOk) {
    DBG_PRINTLN("[SERIAL] waiting for RC:CONNECT handshake before inject");
    return;
  }

  if (line.startsWith("RC:DATA") || line.startsWith("RC:PLOT")) {
    sendLine(line);
    DBG_PRINT("[SERIAL] inject one-shot → phone: ");
```

```
318     DBG_PRINTLN(line.startsWith("RC:DATA") ? "RC:DATA" : "RC:PLOT");
319     return;
320   }
321
322   DBG_PRINT("[SERIAL] unhandled inject (use RC:DATA / RC:PLOT): ");
323   DBG_PRINTLN(line);
324 }
```

Line 305. The factory sketch compiles it, because `ENABLE_SERIAL_INJECT` is 1. If there is no Bluetooth client, or the handshake is not ready yet, it does not send the line. If the line starts with `RC:DATA` or with `RC:PLOT`, it sends it once with `sendLine`. Any other line is only noted on the monitor. The `loop()` timers keep running, so the next automatic packet replaces that line on the phone.

void serialInjectPoll()

```
326 void serialInjectPoll() {
327   while (Serial.available()) {
328     char c = (char)Serial.read();
329     if (c == '\n') {
330       if (serialLineBuf.length() > 0) {
331         serialLineBuf.trim();
332         if (serialLineBuf.length() > 0) {
333           handleSerialInjectLine(serialLineBuf);
334         }
335         serialLineBuf = "";
336       }
337     } else if (c != '\r') {
338       if ((int)serialLineBuf.length() < SERIAL_MAX_LINE) {
339         serialLineBuf += c;
340       } else {
341         serialLineBuf = "";
342         DBG_PRINTLN("[SERIAL] line too long, discarded");
343       }
344     }
345   }
346 }
347 #endif
```

Line 326. Reads the monitor one character at a time up to `\n`. It ignores `\r`, so a line ending of “Both NL & CR” works. When the line closes it trims it and, if text remains, calls `handleSerialInjectLine`. If the buffer passes `SERIAL_MAX_LINE` (192), it throws the buffer away and says so.

```
349 uint16_t stickTo12(int s) {
350   return (uint16_t)(((constrain(s, -100, 100) + 100) * 4095) / 200);
351 }
```

uint16_t stickTo12(int s)

Line 349. Maps a stick from $-100 \ldots 100$ to $0 \ldots 4095$, only for the monitor text. Stick 0 lands near 2048. That number is neither the servo duty nor the `RC:PLOT` byte.

void debugPrintControl(int lx, int ly, int rx, int ry, int lk, int rk, uint8_t sw)

```
353 void debugPrintControl(int lx, int ly, int rx, int ry, int lk, int rk,
    uint8_t sw) {
354   uint16_t lx12 = stickTo12(lx);
355   uint16_t ly12 = stickTo12(ly);
356   uint16_t rx12 = stickTo12(rx);
357   uint16_t ry12 = stickTo12(ry);
358   uint16_t lkU = (uint16_t)constrain(lk, 0, 1023);
359   uint16_t rkU = (uint16_t)constrain(rk, 0, 1023);
360
361   if (lx12 != lastPrintLX) {
362     DBG_PRINTF("L Stick X: %u    ", lx12);
363     DBG_PRINTF("L Stick Y: %u\n", ly12);
364     lastPrintLX = lx12;
365   }
366   if (ly12 != lastPrintLY) {
367     DBG_PRINTF("L Stick X: %u    ", lx12);
368     DBG_PRINTF("L Stick Y: %u\n", ly12);
369     lastPrintLY = ly12;
370   }
371   if (rx12 != lastPrintRX) {
372     DBG_PRINTF("R Stick X: %u    ", rx12);
373     DBG_PRINTF("R Stick Y: %u\n", ry12);
374     lastPrintRX = rx12;
375   }
376   if (ry12 != lastPrintRY) {
377     DBG_PRINTF("R Stick X: %u    ", rx12);
378     DBG_PRINTF("R Stick Y: %u\n", ry12);
379     lastPrintRY = ry12;
380   }
381   if (lkU != lastPrintKnobL) {
382     DBG_PRINTF("Left Knob: %u\n", lkU);
383     lastPrintKnobL = lkU;
384   }
385   if (rkU != lastPrintKnobR) {
386     DBG_PRINTF("Right Knob: %u\n", rkU);
387     lastPrintKnobR = rkU;
388   }
389   if (sw != lastPrintSw) {
390     for (int i = 0; i < 6; i++) {
391       byte bit = (1 << i);
392       if ((sw & bit) != (lastPrintSw & bit)) {
393         DBG_PRINTF("SW%d = %d\n", i + 1, (sw & bit) ? 1 : 0);
394       }
395     }
396     lastPrintSw = sw;
397   }
398 }
```

Line 353. Runs each stick through `stickTo12` and each knob through a 0–1023 limit. It prints an axis only when that value changed from `lastPrint*`, so a still stick does not fill

the monitor. If one of the first six bits of `sw` changes, it prints `SW1` through `SW6`. With `ENABLE_DEBUG` at 0 the function still runs and the macros write nothing.

```
400 void debugPrintButton(int id) {
401   DBG_PRINTF("BTN%d : Pressed\n", id);
402 }
```

void debugPrintButton(int id)

Line 400. Prints `BTNn : Pressed` with the id that arrived in `RC:BTN`.

```
404 void sendLine(const String& line) {
405   SerialBT.print(line);
406   SerialBT.print('\n');
407 }
```

void sendLine(const String& line)

Line 404. The only send toward the phone. It writes the text and then a newline, through `SerialBT`. The handshake, the NAK, the dashboard, and the plot all pass through here.

```
409 void applyFailSafe() {
410   g_lx = g_ly = g_rx = g_ry = 0;
411   g_lk = g_rk = 512;
412   g_sw = 0;
413   steerServoWriteRx(0);
414   DBG_PRINTLN("[IO] fail-safe");
415   // YOUR CODE HERE - stop motors on
disconnect
416 }
```

void applyFailSafe()

Line 409. Sets the four sticks to 0, the two knobs to 512, and `g_sw` to 0, and writes the servo with `rx` 0 (center). Under the second `YOUR CODE HERE`, stop the motors that `onControl()` started. Otherwise the last throttle stays applied after the phone is gone.

```
418 void resetHandshake() {
419   handshakeOk = false;
420   lastPrintLX = lastPrintLY = lastPrintRX
 = lastPrintRY = 0xFFFF;
421   lastPrintKnobL = lastPrintKnobR = 0
xFFFF;
422   lastPrintSw = 0xFF;
423 }
```

void resetHandshake()

Line 418. Sets `handshakeOk` to false and forgets the last monitor values, so the new session prints them again. It does not move the servo or the outputs: the code that saw the drop already called `applyFailSafe()`.

String getValue(const String& line, const char* key)

```
425 String getValue(const String& line, const char* key) {
426   int colon = line.indexOf(':');
427   if (colon < 0) return "";
428
429   int pos = line.indexOf(',', colon);
430   if (pos < 0) return "";
431   pos++;
432
433   while (pos < (int)line.length()) {
434     int keyEnd = line.indexOf(',', pos);
435     if (keyEnd < 0) break;
```

```
436
437     String k = line.substring(pos, keyEnd);
438     int valEnd = line.indexOf(',', keyEnd + 1);
439     String v = (valEnd < 0)
440       ? line.substring(keyEnd + 1)
441       : line.substring(keyEnd + 1, valEnd);
442
443     if (k == key) return v;
444     pos = (valEnd < 0) ? line.length() : valEnd + 1;
445   }
446   return "";
447 }
```

Line 425. Pulls the value of a key from a line `APP:TYPE,key,value,key,value`. It skips the head up to the first comma and walks the pairs. The value runs to the next comma, or to the end of the line on the last pair. Keys are case-sensitive (`lx`, which is distinct from `LX`). A missing key returns an empty string, and `toInt()` of that string is 0.

void handleConnect(const String& app, const String& line)

```
449 void handleConnect(const String& app, const String& line) {
450   if (!app.equalsIgnoreCase(APP_PREFIX)) {
451     sendLine(String("RC:NAK,reason,app_mismatch,expected,") +
APP_PREFIX +
452              ",actual," + app);
453     handshakeOk = false;
454     return;
455   }
456
457   String proto = getValue(line, "proto");
458   if (proto.length() == 0) {
459     sendLine(String("RC:NAK,reason,proto_mismatch,expected,") +
PROTO_WIRE +
460              ",actual,unknown");
461     handshakeOk = false;
462     return;
463   }
464
465   if (proto.equalsIgnoreCase(PROTO_WIRE)) {
466     sendLine("RC:ACK,app,RC");
467     handshakeOk = true;
468     DBG_PRINTLN("[HS] ACK - session ready");
469     return;
470   }
471
472   sendLine(String("RC:NAK,reason,proto_mismatch,expected,") +
PROTO_WIRE +
473            ",actual," + proto);
474   handshakeOk = false;
475 }
```

Line 449. Requires two things, ignoring case. The name before the colon has to be `APP_PREFIX` (`RC`). The `proto` value has to be `PROTO_WIRE` (`simple`). If the name does

not match, it answers RC:NAK with app_mismatch and leaves handshakeOk false. If proto is missing, or carries another value, it answers proto_mismatch. If both match, it sends exactly RC:ACK,app,RC and sets handshakeOk true.

void handleLine(const String& line)

```
477 void handleLine(const String& line) {
478   int colon = line.indexOf(':');
479   if (colon <= 0) return;
480
481   String app = line.substring(0, colon);
482   String rest = line.substring(colon + 1);
483   int comma = rest.indexOf(',');
484   String type = (comma < 0) ? rest : rest.substring(0, comma);
485
486   if (type.equalsIgnoreCase("CONNECT")) {
487     handleConnect(app, line);
488     return;
489   }
490
491   if (!handshakeOk) return;
492   if (!app.equalsIgnoreCase("RC")) return;
493
494   if (type.equalsIgnoreCase("CTRL")) {
495     g_lx = getValue(line, "lx").toInt();
496     g_ly = getValue(line, "ly").toInt();
497     g_rx = getValue(line, "rx").toInt();
498     g_ry = getValue(line, "ry").toInt();
499     g_lk = getValue(line, "lk").toInt();
500     g_rk = getValue(line, "rk").toInt();
501     String swHex = getValue(line, "sw");
502     g_sw = (uint8_t)strtol(swHex.c_str(), nullptr, 16);
503
504     onControl(g_lx, g_ly, g_rx, g_ry, g_lk, g_rk, g_sw);
505   } else if (type.equalsIgnoreCase("BTN")) {
506     onButton(getValue(line, "id").toInt());
507   }
508 }
```

Line 477. If the line has no colon, it returns. It splits the app and the type (the text up to the first comma). CONNECT calls handleConnect even when the handshake is not ready yet. Any other type is ignored while handshakeOk is false, and also when the app is not RC. CTRL stores lx, ly, rx, ry, lk, rk, and sw (the sw field is read in hexadecimal) and calls onControl. BTN calls onButton with the id. A DATA, PLOT, or SET line that arrives from the phone does nothing: this sketch sends those lines, and it does not obey them.

2.2 In TelemetryChannels.h

The file declares the types the three functions use. TelKind is the source of the row: vehicle, ADC, GPIO, or PWM. TelVehicle is the built-in signal (TEL_TEMP, TEL_THROTTLE, TEL_BATT, TEL_STEER, TEL_LIGHTS, TEL_KNOB_L, TEL_KNOB_R, TEL_RY). TelChannel stores the kind, the signal, the pin, the LEDC channel, and the duty range. TelVehicleFn is the pointer to telemetryVehicle().

uint8_t telemetryReadPwm(const TelChannel& ch)

```
46 inline uint8_t telemetryReadPwm(const TelChannel& ch) {
47   if (ch.dutyMax <= ch.dutyMin) return 0;
48   uint32_t duty = 0;
49 #if ESP_ARDUINO_VERSION_MAJOR >= 3
50   if (ch.pin >= 0) duty = ledcRead((uint8_t)ch.pin);
51 #else
52   if (ch.ledcChannel >= 0) duty = ledcRead((uint8_t)ch.ledcChannel);
53 #endif
54   long scaled = constrain((long)duty, ch.dutyMin, ch.dutyMax);
55   return (uint8_t)map(scaled, ch.dutyMin, ch.dutyMax, 0, 255);
56 }
```

Line 46. If `dutyMax` is not greater than `dutyMin`, it returns 0. It reads the duty that is already running: on core 3.x with `ledcRead` of the pin number, on 2.x with `ledcRead` of the channel number. It clips that duty to `dutyMin`…`dutyMax` and scales it to 0–255. The factory servo row uses 204…409, which is the 1000–2000 µs travel; the center comes out near 128. The pin has to be attached in `setup` before this function reads it.

uint8_t telemetryReadOne(const TelChannel& ch, TelVehicleFn vehicle)

```
58 inline uint8_t telemetryReadOne(const TelChannel& ch, TelVehicleFn
vehicle) {
59   switch (ch.kind) {
60     case TEL_ADC:
61       if (ch.pin < 0) return 0;
62       return (uint8_t)map(constrain(analogRead(ch.pin), 0, 4095), 0,
4095, 0, 255);
63     case TEL_GPIO:
64       if (ch.pin < 0) return 0;
65       return digitalRead(ch.pin) ? 255 : 0;
66     case TEL_PWM:
67       return telemetryReadPwm(ch);
68     default:
69       return vehicle ? vehicle(ch.vehicle) : 0;
70   }
71 }
```

Line 58. One byte, according to `ch.kind`. `TEL_ADC` reads the pin and scales 0–4095 to 0–255; a negative pin returns 0. `TEL_GPIO` returns 255 or 0 from `digitalRead`; an output pin reports the level that was written. `TEL_PWM` calls `telemetryReadPwm`. Any other kind calls `vehicle(ch.vehicle)`. If that pointer is null, it returns 0.

```
73 inline void telemetryReadAll(uint8_t out
   [8], const TelChannel table[8],
   TelVehicleFn vehicle) {
74   for (int i = 0; i < 8; i++) out[i] =
   telemetryReadOne(table[i], vehicle);
75 }
```

void telemetryReadAll()

Line 73. Walks the eight rows of `kTelemetryChannels[]` and writes `out[0]` through `out[7]`. `loop()` sends those bytes as `v0` through `v7`. Row 0 is the phone's CH1 and row 7 is CH8. This function has eight slots.

Annex A TeleCon_Classic_Simple.ino

This is the whole sketch, the file the IDE opens. The line numbers are the file's line numbers.

```
1  /*
2   * TeleCon RC Vehicle Pro - Classic Simple (ESP32 Dev Module, no camera)
3   *
4   * Android (TeleCon4ESP32):
5   *   1. Pair Bluetooth "ESP32-TC-RC-BT-Simple" (PIN 1234 if asked)
6   *   2. Control Panel or RC Vehicle Pro → Board DevKit → Classic + Simple
7   *   3. Connect - phone sends RC:CONNECT,proto,simple
8   *
9   * Single-file starter for Normal users. Edit onControl() / onButton().
10  * Advanced Classic Binary: ../../Classic_Binary/
11  */
12
13 #include "BluetoothSerial.h"
14 #include "TelemetryChannels.h"
15
16 // === CONFIG ===
17 #define BT_NAME             "ESP32-TC-RC-BT-Simple"
18 #define BT_PIN              "1234"
19 #define APP_PREFIX          "RC"
20 #define PROTO_WIRE          "simple"
21 #define SERIAL_BAUD         115200
22
23 // 1 = Serial banners + stick/HS/IO logs. Set 0 when Connect works.
24 #define ENABLE_DEBUG        1
25
26 #if ENABLE_DEBUG
27   #define DBG_PRINT(x)      Serial.print(x)
28   #define DBG_PRINTLN(x)    Serial.println(x)
29   #define DBG_PRINTF(...)   Serial.printf(__VA_ARGS__)
30 #else
31   #define DBG_PRINT(x)      ((void)0)
32   #define DBG_PRINTLN(x)    ((void)0)
33   #define DBG_PRINTF(...)   ((void)0)
34 #endif
35
36 #define TELEMETRY_DATA_MS   500
37 #define ENABLE_PLOT         1
38 #define PLOT_MS             50
39
40 #define ENABLE_SERIAL_INJECT 1
41 #define SERIAL_MAX_LINE     192
42
43 // Demo HUD values (replace with ADC reads when you wire sensors)
44 #define DEMO_MOTOR_TEMP     90
45 #define DEMO_BATT_PCT       76
46
47 // === BLUETOOTH ===
48 BluetoothSerial SerialBT;
```

```
49
50  bool handshakeOk = false;
51  bool btWasConnected = false;
52
53  int g_lx = 0, g_ly = 0, g_rx = 0, g_ry = 0;
54  int g_lk = 512, g_rk = 512;
55  uint8_t g_sw = 0;
56
57  unsigned long lastDataMs = 0;
58  unsigned long lastPlotMs = 0;
59
60  #if ENABLE_SERIAL_INJECT
61  static String serialLineBuf;
62  #endif
63
64  static uint16_t lastPrintLX = 0xFFFF;
65  static uint16_t lastPrintLY = 0xFFFF;
66  static uint16_t lastPrintRX = 0xFFFF;
67  static uint16_t lastPrintRY = 0xFFFF;
68  static uint16_t lastPrintKnobL = 0xFFFF;
69  static uint16_t lastPrintKnobR = 0xFFFF;
70  static uint8_t lastPrintSw = 0xFF;
71
72  String getValue(const String& line, const char* key);
73  void handleLine(const String& line);
74  void handleConnect(const String& app, const String& line);
75  void sendLine(const String& line);
76  void resetHandshake();
77  void applyFailSafe();
78  void debugPrintControl(int lx, int ly, int rx, int ry, int lk, int rk, uint8_t sw);
79  void debugPrintButton(int id);
80  uint16_t stickTo12(int s);
81  #if ENABLE_SERIAL_INJECT
82  void serialInjectPoll();
83  void handleSerialInjectLine(const String& line);
84  #endif
85
86
87  // === STEERING SERVO ===
88  // DevKit: set ENABLE_STEER_SERVO 1 and wire a servo on PIN_STEER_SERVO.
89  // rx = 0 is always 1500 µs. Phone trim already offsets rx before it is sent.
90  #ifndef ENABLE_STEER_SERVO
91  #define ENABLE_STEER_SERVO 1
92  #endif
93  #ifndef PIN_STEER_SERVO
94  #define PIN_STEER_SERVO 13
95  #endif
96  #ifndef CH_STEER_SERVO
97  #define CH_STEER_SERVO 2
98  #endif
99
100 // Phone CH1...CH8. Change a row to TEL_ADC, TEL_GPIO, or TEL_PWM and set the pin.
101 // Examples:
102 //   { TEL_ADC,  TEL_TEMP,  34, -1, 0, 1 },            // analog pin 34
103 //   { TEL_GPIO, TEL_LIGHTS, 15, -1, 0, 1 },           // digital pin 15
104 //   { TEL_PWM,  TEL_THROTTLE, 25, 0, 0, 255 },        // 8-bit motor PWM on GPIO 25
105 static const TelChannel kTelemetryChannels[8] = {
106   { TEL_VEHICLE, TEL_TEMP,     -1,              -1,   0,   1 },
107   { TEL_VEHICLE, TEL_THROTTLE, -1,              -1,   0,   1 },
108   { TEL_VEHICLE, TEL_BATT,     -1,              -1,   0,   1 },
109   { TEL_VEHICLE, TEL_STEER,    -1,              -1,   0,   1 },
110   { TEL_VEHICLE, TEL_LIGHTS,   -1,              -1,   0,   1 },
111   { TEL_VEHICLE, TEL_KNOB_L,   -1,              -1,   0,   1 },
112   { TEL_VEHICLE, TEL_KNOB_R,   -1,              -1,   0,   1 },
113   { TEL_PWM,     TEL_STEER,    PIN_STEER_SERVO, CH_STEER_SERVO, 204, 409 },
```

```
};

uint8_t telemetryVehicle(TelVehicle id) {
  switch (id) {
    case TEL_TEMP:
      return (uint8_t)DEMO_MOTOR_TEMP;
    case TEL_THROTTLE:
      return (uint8_t)constrain(map(abs(g_ly), 0, 100, 0, 255), 0, 255);
    case TEL_BATT:
      return (uint8_t)constrain(map(DEMO_BATT_PCT, 0, 100, 0, 255), 0, 255);
    case TEL_STEER:
      return (uint8_t)constrain(map(abs(g_rx), 0, 100, 0, 255), 0, 255);
    case TEL_LIGHTS:
      return (g_sw & 0x01) ? 255 : 0;
    case TEL_KNOB_L:
      return (uint8_t)map(constrain(g_lk, 0, 1023), 0, 1023, 0, 255);
    case TEL_KNOB_R:
      return (uint8_t)map(constrain(g_rk, 0, 1023), 0, 1023, 0, 255);
    case TEL_RY:
      return (uint8_t)constrain(map(abs(g_ry), 0, 100, 0, 255), 0, 255);
    default:
      return 0;
  }
}

constexpr int kSteerCenterUs = 1500;
constexpr int kSteerTravelUs = 500;

uint32_t steerDutyFromUs(int us) {
  int pulse = constrain(us, 500, 2500);
  return (uint32_t)((long)pulse * 4095L / 20000L);
}

uint32_t steerDutyFromRx(int rx) {
  int s = constrain(rx, -100, 100);
  int us = (int)map(s, -100, 100, kSteerCenterUs - kSteerTravelUs, kSteerCenterUs +
kSteerTravelUs);
  return steerDutyFromUs(us);
}

#if ENABLE_STEER_SERVO
void steerServoBegin() {
#if ESP_ARDUINO_VERSION_MAJOR >= 3
  ledcAttach(PIN_STEER_SERVO, 50, 12);
  ledcWrite(PIN_STEER_SERVO, steerDutyFromUs(kSteerCenterUs));
#else
  ledcSetup(CH_STEER_SERVO, 50, 12);
  ledcAttachPin(PIN_STEER_SERVO, CH_STEER_SERVO);
  ledcWrite(CH_STEER_SERVO, steerDutyFromUs(kSteerCenterUs));
#endif
}

void steerServoWriteRx(int rx) {
  uint32_t duty = steerDutyFromRx(rx);
#if ESP_ARDUINO_VERSION_MAJOR >= 3
  ledcWrite(PIN_STEER_SERVO, duty);
#else
  ledcWrite(CH_STEER_SERVO, duty);
#endif
}
#else
void steerServoBegin() {}
void steerServoWriteRx(int rx) { (void)rx; }
#endif

```

```
void onControl(int lx, int ly, int rx, int ry, int lk, int rk, uint8_t sw) {
  debugPrintControl(lx, ly, rx, ry, lk, rk, sw);

  // rx = 0 → 1500 µs. Trim ticks on the phone already shift this value.
  steerServoWriteRx(rx);

  // YOUR CODE HERE - drive motors / other actuators from sticks & knobs
  //   ly = throttle (-100..100), rx = steering, rk = pan, sw bit0 = lights
  (void)lx; (void)ly; (void)ry; (void)lk; (void)rk; (void)sw;
}

void onButton(int id) {
  debugPrintButton(id);

  // YOUR CODE HERE - button actions
  if (id == 2) {
    // Horn note - pulse buzzer briefly
  } else if (id == 4) {
    // Center pan note - move camera pan servo to center
  }
}

void setup() {
  Serial.begin(SERIAL_BAUD);
  delay(200);

  steerServoBegin();

  // Legacy PIN pairing only (Android often fails if SSP + setPin are mixed).
  SerialBT.disableSSP();
  SerialBT.setPin(BT_PIN, 4);

  // isMaster=false, disableBLE=true → Classic SPP only
  if (!SerialBT.begin(BT_NAME, false, true)) {
    Serial.println("Bluetooth init failed!");
    while (true) { delay(1000); }
  }

  DBG_PRINTLN();
  DBG_PRINTLN("=== TeleCon RC Vehicle Pro - Classic Simple ===");
  DBG_PRINT("BT name: ");
  DBG_PRINTLN(BT_NAME);
  DBG_PRINTLN("Pair with PIN 1234 if the phone asks.");
  DBG_PRINTLN("Android: RC Vehicle Pro → Board DevKit → Normal → Classic + Simple");
  DBG_PRINTLN("Handshake: RC:CONNECT,proto,simple  →  RC:ACK,app,RC");
  DBG_PRINTLN("Examples:");
  DBG_PRINTLN("  RC:CONNECT,proto,simple");
  DBG_PRINTLN("  RC:CTRL,lx,0,ly,0,rx,0,ry,0,lk,512,rk,512,sw,00");
  DBG_PRINTLN("  RC:BTN,id,1");
  DBG_PRINTLN("Steer: rx=0 is 1500 µs. Trim stays on the phone.");
  DBG_PRINTLN("Edit onControl() / onButton() at YOUR CODE HERE.");
  DBG_PRINTLN("Telemetry: RC:DATA every 500 ms (speed×10 from |ly|, demo batt/temp).");
#if ENABLE_PLOT
  DBG_PRINTLN("Plot ON - RC:PLOT ~50 ms, CH1...CH8 (edit kTelemetryChannels[]).");
#else
  DBG_PRINTLN("Plot OFF - set ENABLE_PLOT 1 to send RC:PLOT.");
#endif
#if ENABLE_SERIAL_INJECT
  DBG_PRINTLN("Serial inject @ 115200 (after handshake, line ending = Newline):");
  DBG_PRINTLN("  RC:DATA,left,186,right,0,lo,1,ro,0,lg,1,rg,0,analog,90,batt,76,led,01");
  DBG_PRINTLN("  RC:PLOT,v0,128,v1,200,v2,64,v3,180,v4,0,v5,0,v6,0,v7,0");
  DBG_PRINTLN("  Inject is one-shot (pulse); auto telemetry resumes after.");
#endif
  DBG_PRINTLN();
}
```

```
243
244 void loop() {
245   bool connected = SerialBT.hasClient();
246   if (connected != btWasConnected) {
247     btWasConnected = connected;
248     if (connected) {
249       resetHandshake();
250       DBG_PRINTLN("[BT] connected - waiting for RC:CONNECT,proto,simple");
251     } else {
252       applyFailSafe();
253       resetHandshake();
254       DBG_PRINTLN("[BT] disconnected");
255     }
256   }
257
258   while (SerialBT.available()) {
259     String line = SerialBT.readStringUntil('\n');
260     line.trim();
261     if (line.length() > 0) {
262       handleLine(line);
263     }
264   }
265
266 #if ENABLE_SERIAL_INJECT
267   serialInjectPoll();
268 #endif
269
270   if (!handshakeOk) return;
271
272   unsigned long now = millis();
273
274   if (now - lastDataMs >= TELEMETRY_DATA_MS) {
275     lastDataMs = now;
276
277     int speedX10 = abs(g_ly) * 25 / 10;
278     int analog = DEMO_MOTOR_TEMP;
279     int batt = DEMO_BATT_PCT;
280
281     char data[128];
282     snprintf(data, sizeof(data),
283       "RC:DATA,left,%d,right,0,lo,1,ro,0,lg,1,rg,0,analog,%d,batt,%d,led,%02X",
284       speedX10, analog, batt, g_sw);
285     sendLine(data);
286   }
287
288 #if ENABLE_PLOT
289   if (now - lastPlotMs >= PLOT_MS) {
290     lastPlotMs = now;
291
292     uint8_t sample[8];
293     telemetryReadAll(sample, kTelemetryChannels, telemetryVehicle);
294     char plot[128];
295     snprintf(plot, sizeof(plot),
296       "RC:PLOT,v0,%u,v1,%u,v2,%u,v3,%u,v4,%u,v5,%u,v6,%u,v7,%u",
297       sample[0], sample[1], sample[2], sample[3],
298       sample[4], sample[5], sample[6], sample[7]);
299     sendLine(plot);
300   }
301 #endif
302 }
303
304 #if ENABLE_SERIAL_INJECT
305 void handleSerialInjectLine(const String& line) {
306   if (!SerialBT.hasClient()) {
307     DBG_PRINTLN("[SERIAL] no BT client - line not sent");
```

```
    return;
  }
  if (!handshakeOk) {
    DBG_PRINTLN("[SERIAL] waiting for RC:CONNECT handshake before inject");
    return;
  }

  if (line.startsWith("RC:DATA") || line.startsWith("RC:PLOT")) {
    sendLine(line);
    DBG_PRINT("[SERIAL] inject one-shot → phone: ");
    DBG_PRINTLN(line.startsWith("RC:DATA") ? "RC:DATA" : "RC:PLOT");
    return;
  }

  DBG_PRINT("[SERIAL] unhandled inject (use RC:DATA / RC:PLOT): ");
  DBG_PRINTLN(line);
}

void serialInjectPoll() {
  while (Serial.available()) {
    char c = (char)Serial.read();
    if (c == '\n') {
      if (serialLineBuf.length() > 0) {
        serialLineBuf.trim();
        if (serialLineBuf.length() > 0) {
          handleSerialInjectLine(serialLineBuf);
        }
        serialLineBuf = "";
      }
    } else if (c != '\r') {
      if ((int)serialLineBuf.length() < SERIAL_MAX_LINE) {
        serialLineBuf += c;
      } else {
        serialLineBuf = "";
        DBG_PRINTLN("[SERIAL] line too long, discarded");
      }
    }
  }
}
#endif

uint16_t stickTo12(int s) {
  return (uint16_t)(((constrain(s, -100, 100) + 100) * 4095) / 200);
}

void debugPrintControl(int lx, int ly, int rx, int ry, int lk, int rk, uint8_t sw) {
  uint16_t lx12 = stickTo12(lx);
  uint16_t ly12 = stickTo12(ly);
  uint16_t rx12 = stickTo12(rx);
  uint16_t ry12 = stickTo12(ry);
  uint16_t lkU = (uint16_t)constrain(lk, 0, 1023);
  uint16_t rkU = (uint16_t)constrain(rk, 0, 1023);

  if (lx12 != lastPrintLX) {
    DBG_PRINTF("L Stick X: %u    ", lx12);
    DBG_PRINTF("L Stick Y: %u\n", ly12);
    lastPrintLX = lx12;
  }
  if (ly12 != lastPrintLY) {
    DBG_PRINTF("L Stick X: %u    ", lx12);
    DBG_PRINTF("L Stick Y: %u\n", ly12);
    lastPrintLY = ly12;
  }
  if (rx12 != lastPrintRX) {
    DBG_PRINTF("R Stick X: %u    ", rx12);
```

```
    DBG_PRINTF("R Stick Y: %u\n", ry12);
    lastPrintRX = rx12;
  }
  if (ry12 != lastPrintRY) {
    DBG_PRINTF("R Stick X: %u    ", rx12);
    DBG_PRINTF("R Stick Y: %u\n", ry12);
    lastPrintRY = ry12;
  }
  if (lkU != lastPrintKnobL) {
    DBG_PRINTF("Left Knob: %u\n", lkU);
    lastPrintKnobL = lkU;
  }
  if (rkU != lastPrintKnobR) {
    DBG_PRINTF("Right Knob: %u\n", rkU);
    lastPrintKnobR = rkU;
  }
  if (sw != lastPrintSw) {
    for (int i = 0; i < 6; i++) {
      byte bit = (1 << i);
      if ((sw & bit) != (lastPrintSw & bit)) {
        DBG_PRINTF("SW%d = %d\n", i + 1, (sw & bit) ? 1 : 0);
      }
    }
    lastPrintSw = sw;
  }
}

void debugPrintButton(int id) {
  DBG_PRINTF("BTN%d : Pressed\n", id);
}

void sendLine(const String& line) {
  SerialBT.print(line);
  SerialBT.print('\n');
}

void applyFailSafe() {
  g_lx = g_ly = g_rx = g_ry = 0;
  g_lk = g_rk = 512;
  g_sw = 0;
  steerServoWriteRx(0);
  DBG_PRINTLN("[IO] fail-safe");
  // YOUR CODE HERE - stop motors on disconnect
}

void resetHandshake() {
  handshakeOk = false;
  lastPrintLX = lastPrintLY = lastPrintRX = lastPrintRY = 0xFFFF;
  lastPrintKnobL = lastPrintKnobR = 0xFFFF;
  lastPrintSw = 0xFF;
}

String getValue(const String& line, const char* key) {
  int colon = line.indexOf(':');
  if (colon < 0) return "";

  int pos = line.indexOf(',', colon);
  if (pos < 0) return "";
  pos++;

  while (pos < (int)line.length()) {
    int keyEnd = line.indexOf(',', pos);
    if (keyEnd < 0) break;

    String k = line.substring(pos, keyEnd);
```

```
    int valEnd = line.indexOf(',', keyEnd + 1);
    String v = (valEnd < 0)
      ? line.substring(keyEnd + 1)
      : line.substring(keyEnd + 1, valEnd);

    if (k == key) return v;
    pos = (valEnd < 0) ? line.length() : valEnd + 1;
  }
  return "";
}

void handleConnect(const String& app, const String& line) {
  if (!app.equalsIgnoreCase(APP_PREFIX)) {
    sendLine(String("RC:NAK,reason,app_mismatch,expected,") + APP_PREFIX +
             ",actual," + app);
    handshakeOk = false;
    return;
  }

  String proto = getValue(line, "proto");
  if (proto.length() == 0) {
    sendLine(String("RC:NAK,reason,proto_mismatch,expected,") + PROTO_WIRE +
             ",actual,unknown");
    handshakeOk = false;
    return;
  }

  if (proto.equalsIgnoreCase(PROTO_WIRE)) {
    sendLine("RC:ACK,app,RC");
    handshakeOk = true;
    DBG_PRINTLN("[HS] ACK - session ready");
    return;
  }

  sendLine(String("RC:NAK,reason,proto_mismatch,expected,") + PROTO_WIRE +
           ",actual," + proto);
  handshakeOk = false;
}

void handleLine(const String& line) {
  int colon = line.indexOf(':');
  if (colon <= 0) return;

  String app = line.substring(0, colon);
  String rest = line.substring(colon + 1);
  int comma = rest.indexOf(',');
  String type = (comma < 0) ? rest : rest.substring(0, comma);

  if (type.equalsIgnoreCase("CONNECT")) {
    handleConnect(app, line);
    return;
  }

  if (!handshakeOk) return;
  if (!app.equalsIgnoreCase("RC")) return;

  if (type.equalsIgnoreCase("CTRL")) {
    g_lx = getValue(line, "lx").toInt();
    g_ly = getValue(line, "ly").toInt();
    g_rx = getValue(line, "rx").toInt();
    g_ry = getValue(line, "ry").toInt();
    g_lk = getValue(line, "lk").toInt();
    g_rk = getValue(line, "rk").toInt();
    String swHex = getValue(line, "sw");
    g_sw = (uint8_t)strtol(swHex.c_str(), nullptr, 16);
```

```
503
504     onControl(g_lx, g_ly, g_rx, g_ry, g_lk, g_rk, g_sw);
505   } else if (type.equalsIgnoreCase("BTN")) {
506     onButton(getValue(line, "id").toInt());
507   }
508 }
```

Annex B `TelemetryChannels.h`

The header stands on its own. The `.ino` includes it and does not copy it.

```
1  #pragma once
2
3  #include <Arduino.h>
4
5  /*
6   * Phone CH1...CH8 are RC:PLOT v0...v7.
7   * Edit kTelemetryChannels[] in the sketch to choose what each channel reports (0-255).
8   *
9   * TEL_VEHICLE  built-in signal (temp, throttle, battery, steer, lights, knobs)
10  * TEL_ADC      analogRead(pin), 0-4095 scaled to 0-255
11  * TEL_GPIO     digitalRead(pin) → 0 or 255 (an output pin reports what was written)
12  * TEL_PWM      LEDC duty on pin. dutyMin maps to 0, dutyMax maps to 255.
13  *              8-bit motor PWM uses 0...255. 12-bit servo travel 1000-2000 µs uses 204...409.
14  *              Attach that pin with ledcAttach / ledcSetup before it is read.
15  */
16
17 enum TelKind : uint8_t {
18   TEL_VEHICLE = 0,
19   TEL_ADC,
20   TEL_GPIO,
21   TEL_PWM,
22 };
23
24 enum TelVehicle : uint8_t {
25   TEL_TEMP = 0,
26   TEL_THROTTLE,
27   TEL_BATT,
28   TEL_STEER,
29   TEL_LIGHTS,
30   TEL_KNOB_L,
31   TEL_KNOB_R,
32   TEL_RY,
33 };
34
35 struct TelChannel {
36   TelKind kind;
37   TelVehicle vehicle;
38   int pin;
39   int ledcChannel;
40   int dutyMin;
41   int dutyMax;
42 };
43
44 using TelVehicleFn = uint8_t (*)(TelVehicle id);
45
46 inline uint8_t telemetryReadPwm(const TelChannel& ch) {
47   if (ch.dutyMax <= ch.dutyMin) return 0;
48   uint32_t duty = 0;
49 #if ESP_ARDUINO_VERSION_MAJOR >= 3
50   if (ch.pin >= 0) duty = ledcRead((uint8_t)ch.pin);
51 #else
52   if (ch.ledcChannel >= 0) duty = ledcRead((uint8_t)ch.ledcChannel);
```

```
#endif
  long scaled = constrain((long)duty, ch.dutyMin, ch.dutyMax);
  return (uint8_t)map(scaled, ch.dutyMin, ch.dutyMax, 0, 255);
}

inline uint8_t telemetryReadOne(const TelChannel& ch, TelVehicleFn vehicle) {
  switch (ch.kind) {
    case TEL_ADC:
      if (ch.pin < 0) return 0;
      return (uint8_t)map(constrain(analogRead(ch.pin), 0, 4095), 0, 4095, 0, 255);
    case TEL_GPIO:
      if (ch.pin < 0) return 0;
      return digitalRead(ch.pin) ? 255 : 0;
    case TEL_PWM:
      return telemetryReadPwm(ch);
    default:
      return vehicle ? vehicle(ch.vehicle) : 0;
  }
}

inline void telemetryReadAll(uint8_t out[8], const TelChannel table[8], TelVehicleFn vehicle) {
  for (int i = 0; i < 8; i++) out[i] = telemetryReadOne(table[i], vehicle);
}
```